

Open with the thing only you can show. *Then* say what it was — they have heard “the problem in agent payments is...” nine times today.

THE SPINE — SAME EVERY TABLE

- 0:00

“Two AI agents that have never met. One hires the other, pays it, and gets work back — and nothing in the middle can see what the work was. I’m going to make the seller cheat.”

Click. Do not narrate while it runs.
- 0:20

“The seller answered a different job than the one that was ordered. The contract on Base just refused it — MatchFalse — and the payment never settled. He did the work and gets nothing.”

Point at two lines: the rejection code, and payment not settled.
- 1:00

The problem, now that they have watched it get solved. Payment, private execution and reputation are each solved; nothing connects them. You can pay an agent and get an answer – nothing proves it answered your question with your data. One signed commitment travels from the payment, through the enclave, into the reputation index. Intent-bound verification.
- 1:30

The track section – the three boxes below. Sixty seconds, one of them.
- 2:30

What we do not claim. Never skip this – thirty-nine of forty teams are overclaiming and the judges know it. We didn’t build the TEE, OG did. The match check runs in unattested code because we had no TDX host, so the client can verify it and a stranger can’t – that gap is real and it’s what we’d close next. The reputation isn’t Sybil-proof; faking it is expensive and visible, not impossible.
- 2:50

“Everything on that screen is testnet but none of it is mocked — every hash opens on a public explorer. The write-up for your track is in the repo, with the exact files and gates.”

1:30 — THE SIXTY SECONDS THAT CHANGES

OG · BEST AI PRODUCT The compute The model runs in a Sealed Inference enclave, so the party being paid never sees the brief. The output comes back TEE-signed. And here is the part that is ours: the client’s signed commitment is written into the top of the prompt and the model copies it verbatim into the answer. So the signature OG’s hardware produces covers a body that contains the client’s intent. Bob can’t manufacture that on his own machine – the first link comes from OG silicon. The deliverable is then archived on OG Storage, encrypted – anyone can fetch it by root hash, only the client can read it. SHOW ogVerified=true · intentEchoed · the archive root · the pinned provider. IF ASKED, say it plainly: ProviderType: centralized, ProviderIdentity: aliyun. The seal is real; the operator is one cloud.	THE GRAPH · AI USE CASE The read layer Alice is never given Bob’s address. She queries the subgraph by skill and ranks by deliveries a contract confirmed. And you cannot write to this reputation – there is no review endpoint. The only thing that increments the counter is a JobVerified event from the Verifier. The rejected attempts are indexed on the same record, so cheating is permanently visible exactly where the hiring decision gets made. SHOW the discovery panel, then the query endpoint – offer to let them POST the query themselves. THE STRONG DETAIL, with the numbers: 31 agents indexed, 21 distinct owners, 28 of them not ours – strangers register into this public ERC-8004 registry while we watch. The panel shows only 2 because it filters by skill and only 3 registrants fill the skills field in. Say that yourself before they ask.	HEDERA · AGENTIC PAYMENTS The money & the timeline x402 through the blocky402 facilitator – and the split matters. Alice authorises, but the money stays hers. Bob does the work, and only after the contract emits JobVerified does he submit the authorisation. On the HCS topic, the honest run ends ... → OUTPUT_COMMIT → SETTLED. The fraud run stops at OUTPUT_COMMIT. The missing stage is the whole argument – recorded on a chain that isn’t ours to edit. Every stage is a commitment, never content. The brief never touches Hedera. HONEST, UNPROMPTED: this rail buys autonomy and a timestamped trail, not privacy. The stealth recipient is on the Base run – offer the contrast.
---	--	--

BEFORE YOU SIT DOWN

- ☐ /dashboard open, fraud mode already selected, logs cleared
- ☐ Show the archive from the recorded run instead – same evidence, no dead air
- ☐ Second tab: the proof bundle /api/proof?mode=none, already downloaded
- ☐ OG_STORAGE=0 for the live click – the archive adds ~15s to a ~13s run
- ☐ One warm-up run before the first table – broker setup is a first-call cost
- ☐ Subgraph 429? Say so, don't reload – honest failure is the right behaviour

THE FOUR YOU WILL ACTUALLY GET – TWO SENTENCES, THEN STOP

“Did you build the TEE?”
No. OG did. We bind it to the client’s intent. We verify signatures on chain and check the attestation off chain at setup – not a raw on-chain TDX quote.

“Is the match check verifiable by me, or do I have to trust you?”
Today: by the client, not by a stranger. The recompute runs in unattested code because we had no TDX host. With an attested runtime the attestation would read codeDigest + inputHash + outputHash and nobody would need to trust us. That is one deployment away, not a redesign – the binding is already a pure function with no environment reads in it.

“What stops an agent from farming its own reputation?”
Nothing stops it; it costs. You can’t write the score, only earn it, and earning it means a job the contract verified – a real inference and real gas per fake delivery. But a ring of agents you control can hire each other and every job is genuinely valid, because cryptography can’t tell a fake relationship from a real one. What we do is make it paid and visible: every job’s client, timestamp and price is in the same public index, so the ring is queryable. The right fix is weighting a delivery by the client’s own standing – computable from data we already index. We haven’t built it.

“Why four chains?”
Because none of them duplicates another. Base is the verdict, Hedera is the money and the timeline, The Graph is the read layer, OG is the compute. Drop one and something stops being checkable.

“The model output isn’t deterministic — how do you recompute it?”
We don’t. We recompute the intent hash from the brief and the data, which is deterministic, and bind the output by its hash. Nothing here ever tries to reproduce a model’s answer.

“How slow is it?”
p50 21.0s, p95 22.7s over 5 runs against a 60s budget – fixtures/latency/P0-G.json, reproduce with pnpm measure:e2e. The OG inference call is ~12.4s of that and is the dominant term.

LIVE IDS – EVERY ONE OPENS ON A PUBLIC EXPLORER

VERIFIER · BASE SEP.	0x3B116D648B710f551e37223c4c4d39879AFEEb96
OG PROVIDER	0xa48f01287233509FD694a22Bf840225062E67836 · qwen/qwen2.5-omni-7b · TeeML
HEDERA TOPIC	0.0.9738448
ERC-8004	Bob 8429 · Alice 8431 · registry 0x8004A818BFB912233c491871b3d84c89A494BD9e
HOSTED DEMO	mithra-protocol.vercel.app/dashboard
LATENCY	p50 21.0s · p95 22.7s · 5 runs · 60s budget

SENTENCES THAT LOSE THE ROOM

- × “solves prompt injection” – task substitution and input tampering only

× “first confidential agent payments” – ProwlFi and TACEO shipped first

× “two TEEs” – one, OG’s

× “decentralized compute” – the pinned provider is one cloud, aliyun
- × “Sybil-proof” – expensive and visible, not impossible

× “private on Hedera” – that run buys autonomy; privacy is the Base run

× “the OG signature covers the answer” – it covers its fingerprint

× “we built the TEE” – OG did; we bind it to intent